

MIGRATING AIRFLOW AT SCALE WHAT THE DOCS DON'T TELL YOU

A practical guide from prerequisites to production

ASTRONOMER





Who I am

Olivier Daneau

Senior Manager @ Astronomer

4+ Years of Orchestrating Airflow Migrations

Helped dozens of large organizations operate their business on Airflow

Astronomer's Global Services Delivery

Global Delivery Team

40+ delivery managers, architects, and engineers around the world

An extensive **network of partners**

Active **Airflow committers** on staff

100+

Engagements every year

50,000+

Dags Migrated

Agenda

01 The guidance you'll find online

02 What Airflow at scale actually looks like

03 How complexity derails migrations

04 How to build, migrate, and operate at scale

The guidance you'll find online

ASTRONOMER

Simple Migrations - Step by Step

1

Complete the
infrastructure
prerequisites

2

Test Dags
against new
image

3

Provision new
environment

4

Migrate conns,
vars & metadata

5

Copy Dags to
new environment

6

Test Dags &
Validate data

7

Cut over

Assumptions

- One team
- One source and one target
- A limited number of Dags
- The same Airflow version
- One cutover window

Enterprise Airflow rarely looks like this.

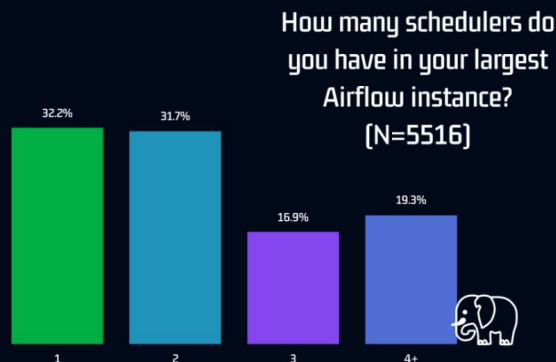
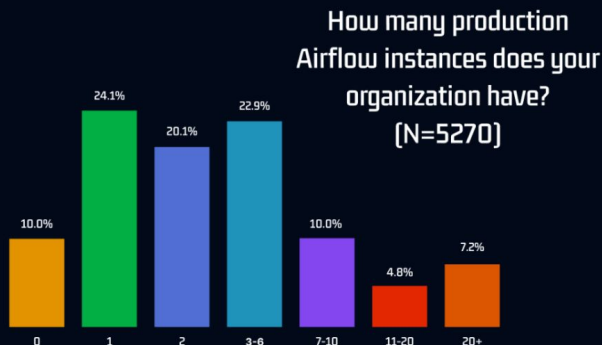
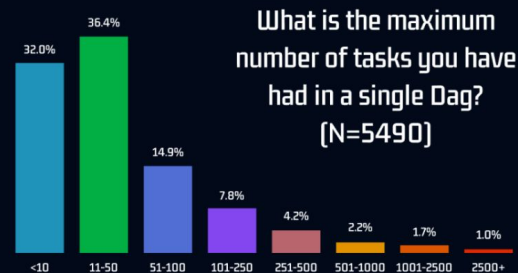
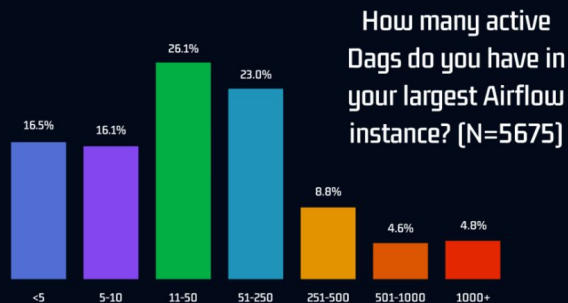
What Airflow at scale actually looks like

What Airflow at scale actually looks like

LET'S DRAW THE LINE AT

250+ Dags

10+ Airflow environments



Source: 2025 Apache Airflow User Survey

Complexity Factors

Teams

Dags

Airflow environments

Git Repos & CI/CD

Engineering practices

Process overhead

Stakeholders

Business operations

How complexity derails migrations

WHEN COMPLEXITY GOES UNMANAGED

TEAMS

Teams start in parallel and repeat each other's mistakes.

DAGS

Scope changes mid-migration and leads to unplanned refactoring.

AIRFLOW ENVIRONMENTS

Airflow, Python, and OS dependencies are pinned manually. Every team repeats the same process from scratch.

GIT REPOS & CI/CD

Fragmented repos and pipelines leave each team maintaining its own path to production.

WHEN COMPLEXITY GOES UNMANAGED

ENGINEERING PRACTICES

Without shared patterns, every team starts from scratch.

PROCESS OVERHEAD

Approvals and IT/Infra prerequisites arrive late. Teams sit blocked.

STAKEHOLDERS

Unclear ownership stalls decisions and fills the calendar with meetings.

BUSINESS OPERATIONS

Untested cutover process turns into a fire drill, a rushed rollback and backfills.

How to build, migrate, and operate at scale

HOW MIGRATION WORKS AT SCALE

BUILD THE PATH

Set the foundation before scaling

Engage networking, security, identity, and DevOps teams early

Define the target architecture, scope and record key decisions

Create reusable templates, automation, readiness criteria, and runbooks

MIGRATE IN WAVES

Prove the path, learn, and repeat

Sequence teams by readiness and train them before each wave

Audit Dags and separate blockers from technical debt

Migrate, test for parity, cut over, stabilize, and improve the process

OPERATE & IMPROVE

Make the new platform sustainable

Establish monitoring, incident response, and SLA tracking

Transfer knowledge and ownership to the teams

Address technical debt and optimize reliability, performance, and cost

KNOW WHAT YOU ARE ACTUALLY MIGRATING

You cannot safely migrate what nobody owns or understands.

- Inventory Dags, Git repos, Python, Linux and other dependencies, identify Dag owners
- Group Dags into categories, look for repeating patterns
- Identify orphaned Dags, undocumented business logic and non-idempotent Dags
- Retire workflows that should not move

SEPARATE MIGRATION FROM MODERNIZATION

See the migration as an opportunity to update your backlog, not to fix everything

- Define what “lift and shift” means before work begins
- Separate platform migration, Airflow upgrades, and Dag refactoring
- Establish a definition of done for each team and migration wave
- Record architecture decisions and scope exceptions

BUILD REUSABLE ARTIFACTS ONCE

Build artifacts once, review and improve them after every wave

Airflow enablement materials

Onboarding questionnaire

Dag Auditing scripts

Git repo templates

CI/CD templates

Infra-as-code templates

Testing and validation
checklists

Cutover and rollback strategy

Migration progress tracker

ASSIGN OWNERSHIP BEFORE EACH WAVE

Define clear ownership early to prevent communication issues and repeated meetings.

- The **migration lead** owns sequencing, dependencies, blocker escalation, and progress tracking.
- The **platform team** owns the platform, standards, automation, and enablement.
- **Dag teams** own code remediation, testing, and business and data validation.
- **Architecture, security, networking**, and **devops** own their respective IT prereqs and approvals.
- Assign **support** escalation responsibilities before scheduling the cutover and go-live dates.

SEQUENCE THE TEAMS INTO WAVES

Use readiness, capacity, complexity, dependencies, and business priority to determine the order.



WAVE 1: PROVE THE PATH



WAVE 2: PROVE REPEATABILITY



SCALE THE PROCESS

WHO TO PICK FIRST?

- Teams with bandwidth and motivation
- Dags with clear ownership and well-understood business logic
- Dags with low levels of debt and customization

HOW TO ONBOARD THEM?

- Train and enable users before each wave
- Provide a migration checklist with action items
- Offer self-serve artifacts and onboarding

WHERE AI HELPS

CODE AND DATA EXPLORATION

Map the Dag code to data being transformed, understand the business logic and how it is used.

DAG AUDITING

Scan the Dags and Airflow environment for compatibility issues, and scope the migration effort.

FAILURE INVESTIGATION

Read logs, run history, and Dag code together to do root cause analysis faster.

DAG AUTHORIZING

Draft a new Dag from a short description, run tests and iterate quickly without writing code.

WORKFLOW TRANSLATION

Convert workflows from other orchestration platforms into Dags easily.

CODE REVIEW

Flag Dag code issues on every pull request, add inline comments, before they merge.

At scale, migration complexity comes less from the technology and more from the organization around it.

BUILD

Resolve prerequisites early

Standardize processes

Templatize & Automate

MIGRATE

Repeat the migration steps for every team

Learn from every wave

OPERATE

Monitor reliability

Address technical debt

Optimize cost & performance

THANK YOU

QUESTIONS?

Olivier Daneau

[linkedin.com/in/odaneau](https://www.linkedin.com/in/odaneau)



SHARE YOUR FEEDBACK

The 2026 Apache Airflow[®] Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or Dag Authoring Certification (normally \$150 each)

